Dimensionality Reduction

ICT 3.3 Machine learning and Data Mining

Emanuela Boros, May 2026

What We Saw Before

Regularization:

- reduces model **complexity**
- penalizes **large weights**
- prevents **overfitting**

What We Saw Before

Regularization:

- reduces model **complexity**
- penalizes **large weights**
- prevents **overfitting**

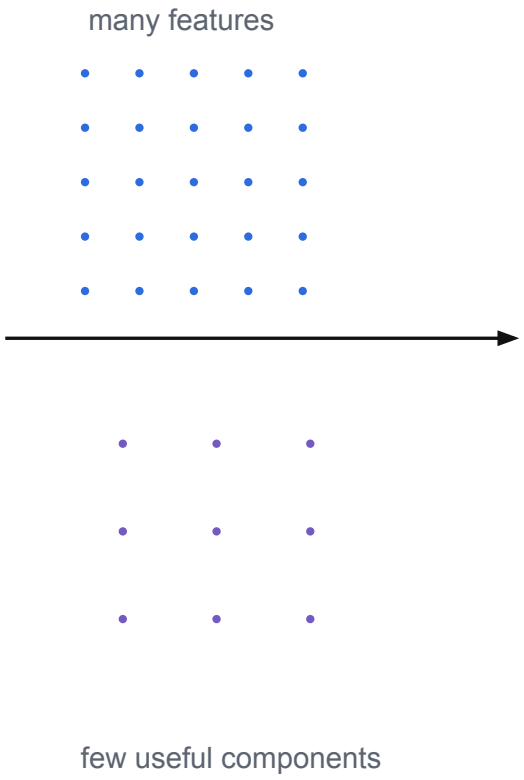
Can we simplify the **data** itself instead of the **model**?

Why Do We Need to Reduce Dimensions?

- Data can have many features
  - many features
  - noise increases

  → harder to generalize

- Hard to visualize
- Noisy variables
- Some features are redundant



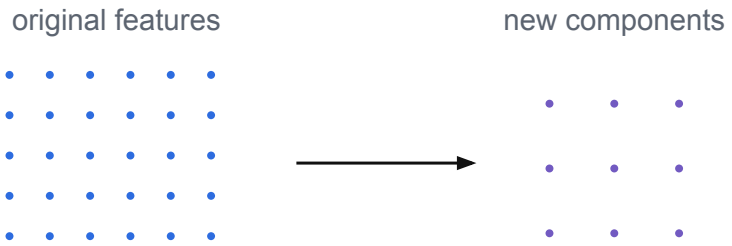
Why Do We Need to Reduce Dimensions?

- Data can have many features
  - many features
  - noise increases

  → harder to generalize

More dimensions = more ways to overfit!

- Hard to visualize
- Noisy variables
- Some features are redundant

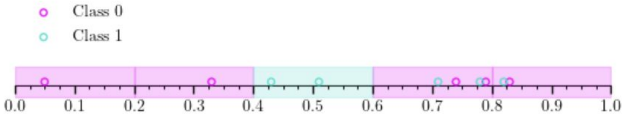


The Curse of Dimensionality

The Curse of Dimensionality

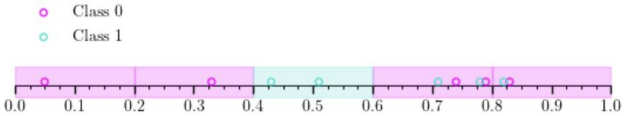
The more features, the higher risk of overfitting is the curse of dimensionality in a nutshell.

The Curse of Dimensionality

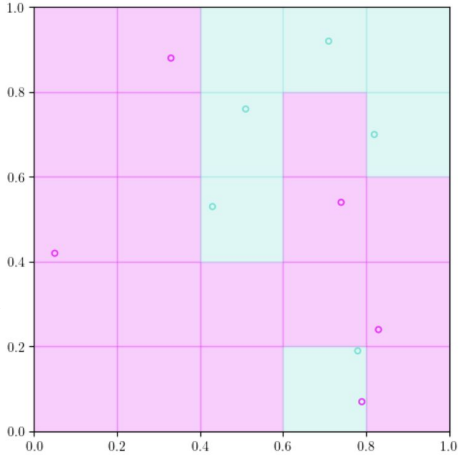


1 feature (ie size of the apartment)

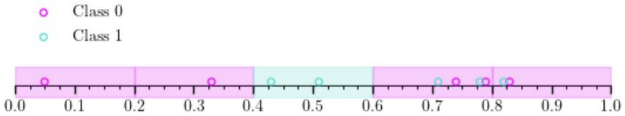
The Curse of Dimensionality



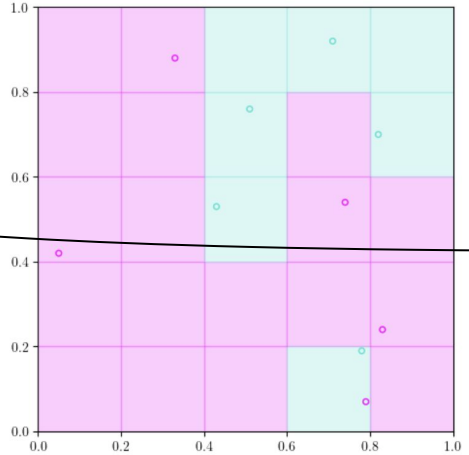
2 features (size of the apartment & number of rooms)

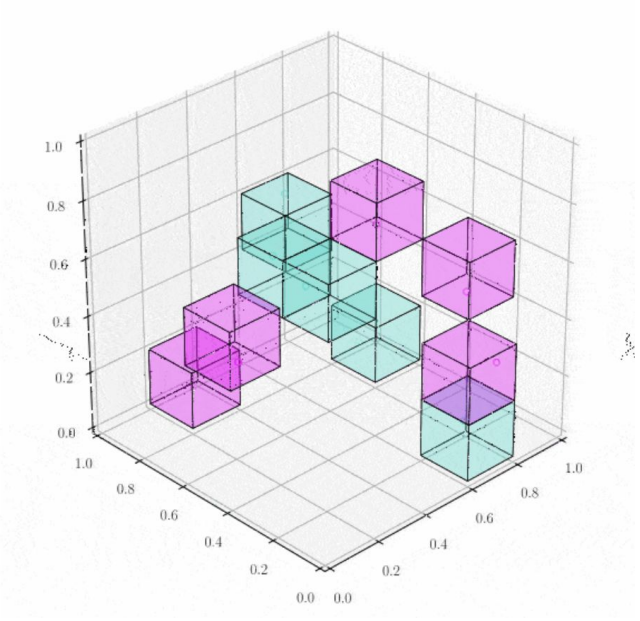


The Curse of Dimensionality



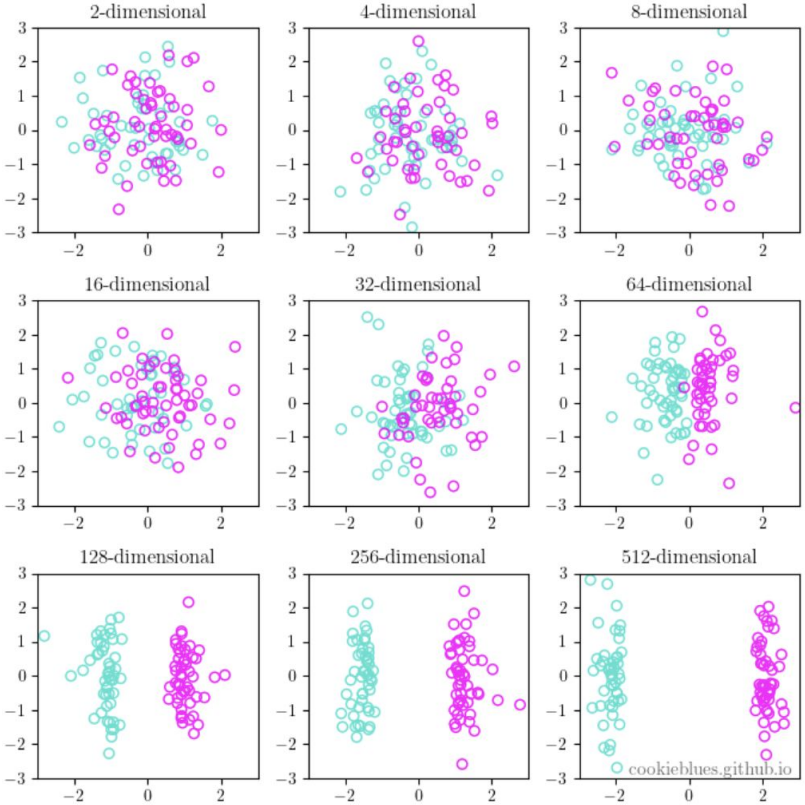
3 features (size of the apartment & number of rooms & living area)





The Curse of Dimensionality

Separation in very high dimensions may be unreliable unless we have enough data.



Dimensionality reduction helps fight sparsity by representing data in a smaller, more useful space.

The Curse of Dimensionality in short

When dimensionality increases, space becomes huge.

- → covering a line is easy.
- Covering a square needs more points.
- Covering a cube needs even more.
- In 100 dimensions, almost everything is empty.

- points become far from each other
- data becomes sparse
- models need many more examples
- noise becomes easier to learn

Why Should We Reduce Dimensions?

Apartment example

- size
- number of rooms
- living area
- price per m^2

These features are related. A bigger apartment usually has more rooms. That means some information is **duplicated**.

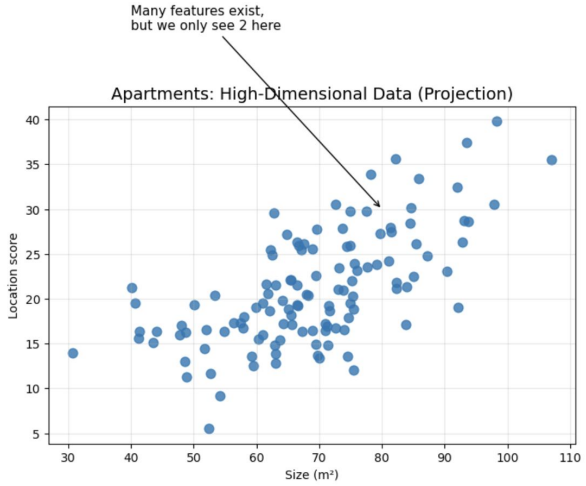
old features

size
rooms
area
age

→

new components

PC1: overall size
PC2: remaining variation



Types of Dimensionality Reduction Methods

Linear methods

PCA, LDA

→ find straight directions / subspaces

Non-linear methods

t-SNE, UMAP

→ capture curved or complex structures

PCA (Principal Component Analysis)

1 Redundancy

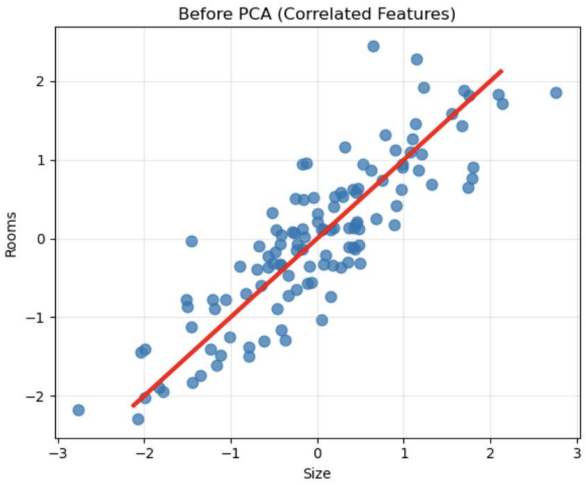
- duplicate information
- unnecessary complexity

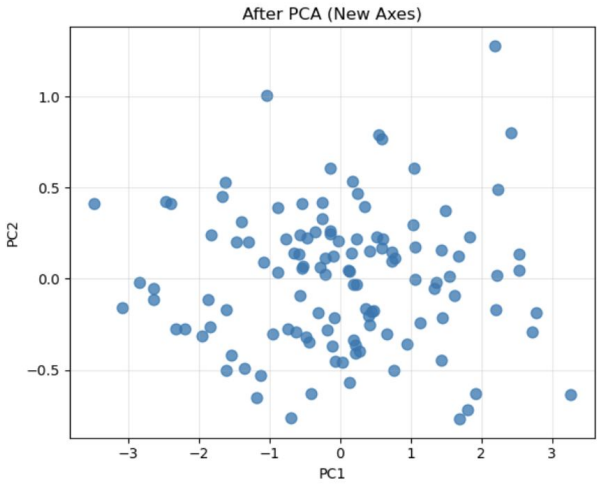
2 Harder for models

- unstable weights
- harder to interpret

3 Overfitting (again)

- model can exploit noise in redundant features





PCA

- size of apartment
- living area

→ both help predict price → both are similar → **correlation**

PCA removes this correlation, but it is not losing information, just reorganizes it. Dimensionality reduction finds a simpler space that **preserves the important structure** of the data.

$$y = w_1 \cdot \text{size} + w_2 \cdot \text{rooms}$$

$$y \approx w_1 \cdot \text{size} + w_2 \cdot \frac{\text{size}}{20}$$

$$y \approx \left(w_1 + \frac{w_2}{20}\right) \cdot \text{size}$$

- many ways to represent the same solution
- no unique answer

$$\left.\begin{aligned} w_1 = 2, w_2 = 0 \\ w_1 = 1, w_2 = 20 \\ w_1 = 3, w_2 = -20 \end{aligned}\right\} y \approx 2 \cdot \text{size}$$

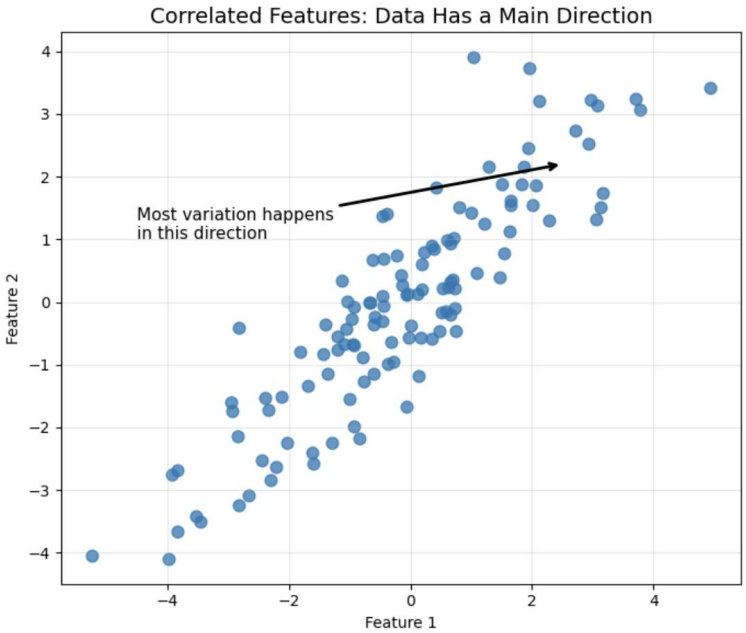
Different weights, same prediction

PCA

We want to simplify the data without losing important information.

- data is not random
- it has a direction
- some directions matter more

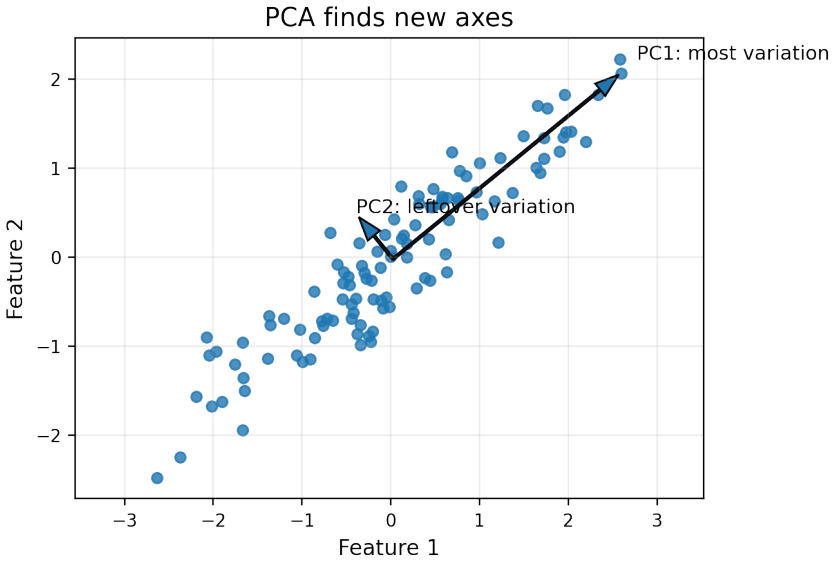
PCA finds this main direction automatically.



PCA

Question PCA asks:

"If I look at this cloud from one straight direction, where do I see the most spread?"

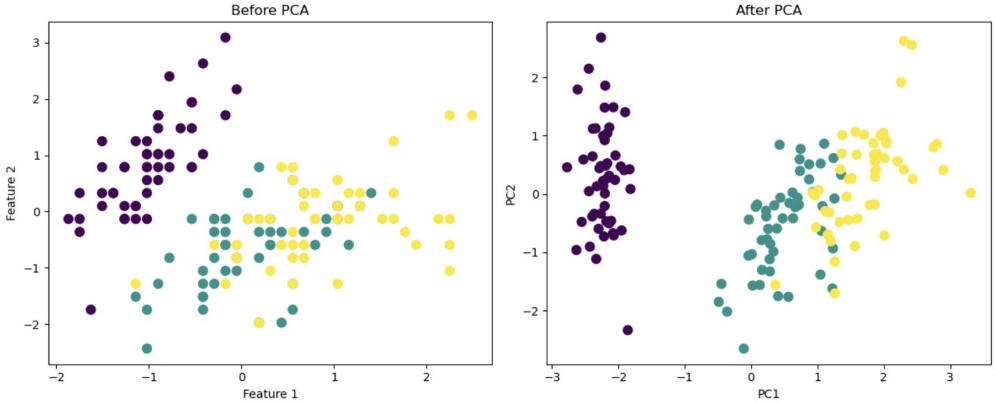


- PC1 = strongest direction
- PC2 = next strongest direction, perpendicular to PC1
- keep first k components, drop the rest

PCA

- Unsupervised method
- Does not use labels
- Finds directions where data varies the most
- Useful for compression, visualization, noise reduction

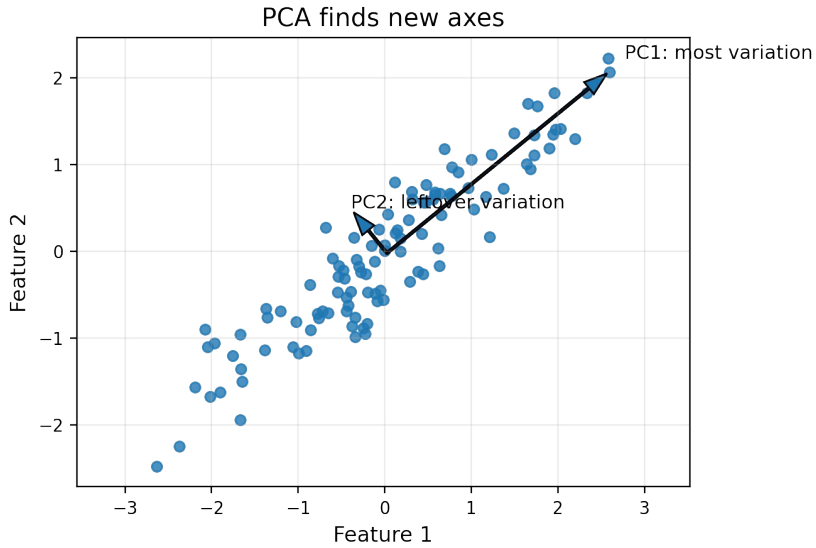
→ PCA rotates the data so that the most important direction becomes the first axis.

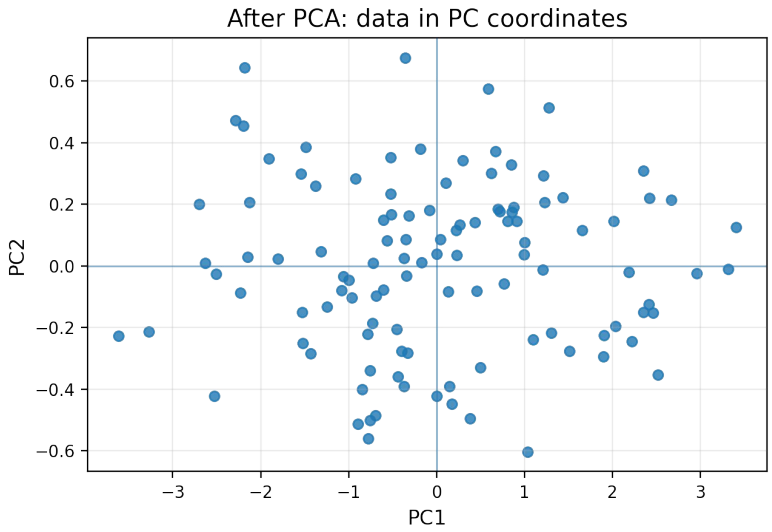


$$\max_{\mathbf{w}}; \mathrm{Var}(X\mathbf{w})$$

PCA

PCA will create a smaller of new features that are a linear combination of the larger number of original features.





Before: original features can be correlated.

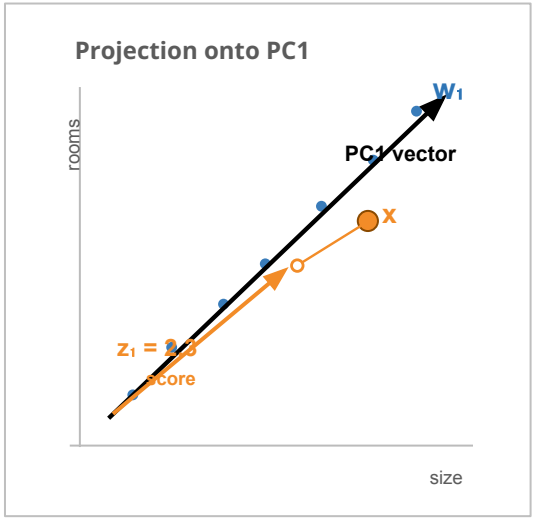
After: PC1 and PC2 are new, uncorrelated axes.

PCA

1 PC1 direction = vector

$w_1 = [0.9, 0.4]$

The direction of the new PCA.
It says how original features are combined.
Example: mostly size + some rooms.



2 PC1 score = number

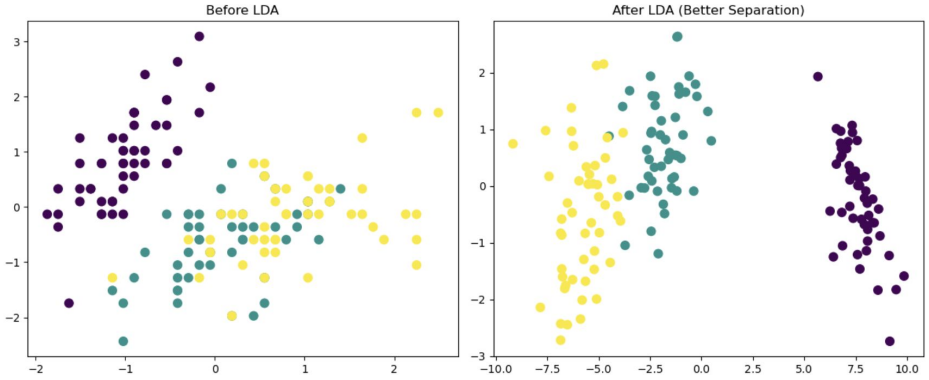
$z_1 = (x - \mu) \cdot w_1$

One value for one data point.
Example: $z_1 = 2.3$ means the point is 2.3 units along the PC1.

Keeping only PC1: **[size, rooms] → z_1** replace two correlated features with one coordinate along the main direction.

LDA (Linear Discriminant Analysis)

- Supervised method
- Uses labels
- Finds directions that separate classes
- Useful for classification and visualization



Intuition:
PCA asks: *Where is the data most spread out?*
LDA asks: *Where are the classes best separated?*

LDA keeps directions that best separate labels.

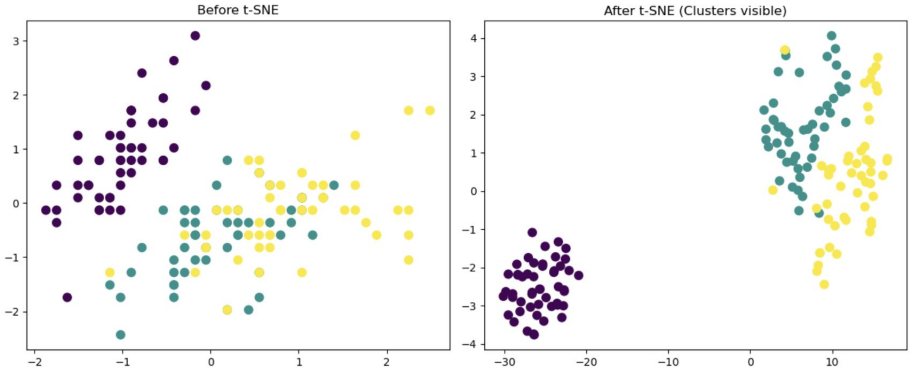
$$\max_{\mathbf{w}}; \frac{\text{between-class variance}}{\text{within-class variance}}$$

t-SNE (t-distributed Stochastic Neighbor Embedding)

t-SNE: t-distributed Stochastic Neighbor Embedding

- Non-linear method
- Mostly used for visualization
- Preserves local neighborhoods
- Similar points stay close together

→ t-SNE tries to keep “neighbors” close when moving from high dimensions to 2D.



$$\min; KL(P \parallel Q)$$

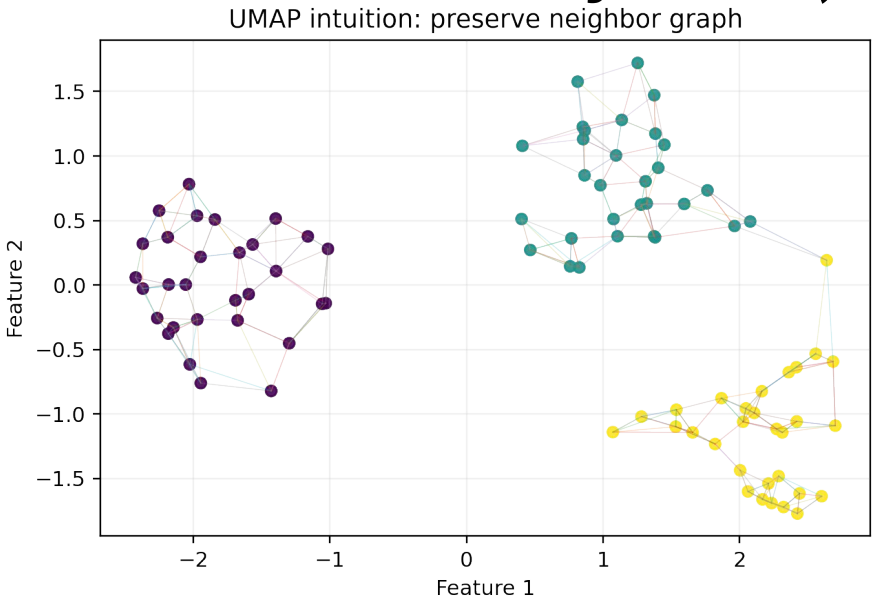
t-SNE is good for seeing clusters, not for measuring exact distances.

UMAP (Uniform Manifold Approximation and Projection)

- Non-linear method
- Faster than t-SNE
- Preserves local and some global structure
- Good for large datasets

→ Build a graph where every point connects to its nearest neighbors. Then create a 2D map that preserves this graph.

UMAP is often used to visualize complex high-dimensional data efficiently.



Warning: UMAP plots are visual explanations, not perfect maps of reality.

Which Method Should I Use?

Start simple; add complexity only when it helps.

Method	Best for	Uses labels?	Advice
PCA	compression, noise reduction	No	first method to try
LDA	class separation	Yes	use when labels matter
t-SNE	cluster visualization	No	do not read distances too literally
UMAP	large visualization	No	good with many embeddings

NLP

Practical rule: for teaching and debugging, use PCA first. For visualizing clusters, try UMAP or t-SNE. For prediction with labels, compare PCA with LDA or no reduction.

Other examples

Images

Each pixel is a feature. A 28×28 image has 784 dimensions. PCA can reduce it to 2D for visualization.

Documents

Each word can be a feature. Topic modeling reduces many word features to a few themes.

many features

[0, 1, 0, 0, 5, 2, 0, ...]
[1, 0, 3, 0, 0, 7, 0, ...]
[0, 0, 0, 4, 1, 0, 8, ...]

→

few components

PC1
PC2

Clustering

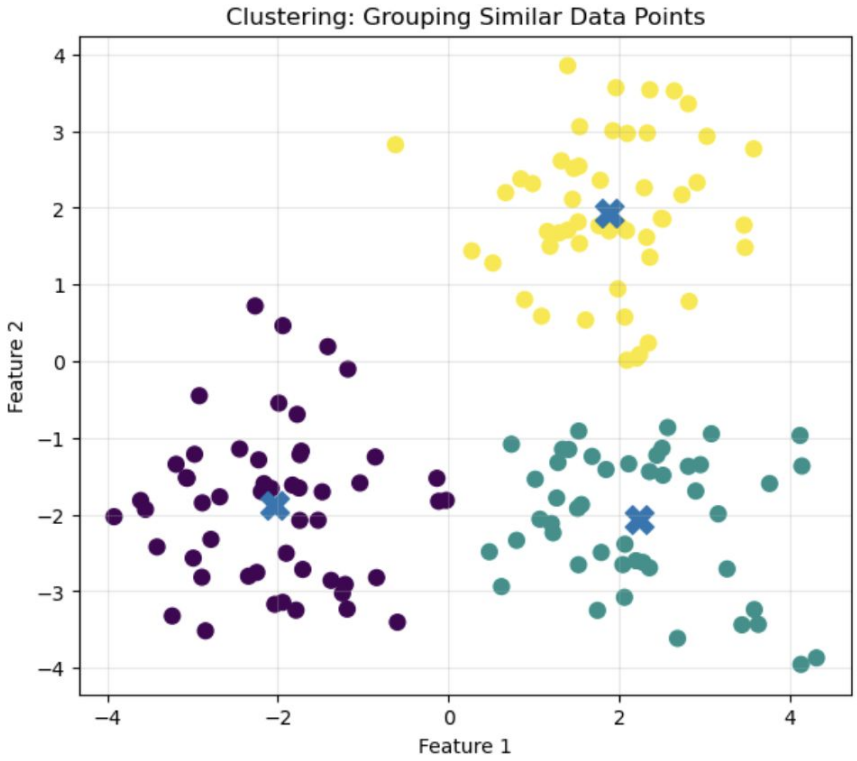
Group similar data points together

Examples

- group customers
- group images
- group documents

Methods

- K-means (can use PCA)
- Hierarchical clustering



Topic Modelling

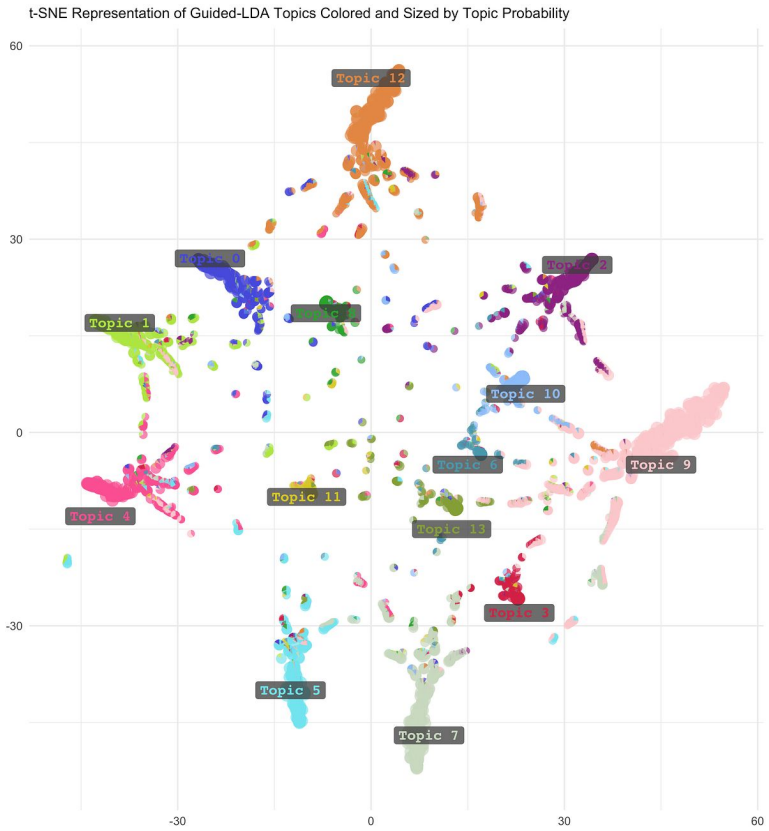
Discover hidden themes (topics) in a collection of documents

- documents = high-dimensional (many words)
- reduce to a few topics

📄 Document → “economy, market, inflation” → Topic: Economics

Methods

- LSA (uses PCA)
- LDA



Mini Assignment

Clone the repo!

What you need:

- GitHub account

TODOs:

- Exercise notebook